Tracking Columns With Sensitive Information

To maintain a list of columns with sensitive values, you can track that information within PostgreSQL.

PostgreSQL gives users the ability to add database-level comments to tables and fields. These comments are then dumped just like other structural details when pg_dump runs. You'll use the column commenting capability to track your sensitive columns.

Connect to the rideshare_development database from psql and run this SQL statement:

sql/create_comment_for_table_column.sql

```
COMMENT ON COLUMN users.email IS 'sensitive_data=true';
```

The statement adds a comment to the users.email field using a simple key and value structure to mark a sensitive field.

To confirm the comment was added, run \d+ users. This expanded form of \d includes additional information like comments. If the output from \d+ users is difficult to read, try running \pset format wrapped to toggle wrapping and run again.

To the very right of the email column you should now see sensitive_data=true as the column comment.

For each sensitive field you've identified, create this type of comment.

To keep all databases in sync, create the database comments using a Rails Migration app. Database comments will then be included in db/structure.sql and become part of your database structure.

Now that you know you've started to identify and track columns to scrub, how will you perform the scrubbing?

Comparing Direct Updates and Table Copying Strategies

For your scrubbing system you'll use two techniques which can be called "Direct Updates" and "Table Copying."

Direct Updates are simply a SQL UPDATE statement that overwrites column values. Direct Updates are a strategy recommended for smaller tables with fewer than one million rows, because UPDATEs can become slow on very large tables.